

## Part 1: C Interview Questions & Answers (1–25)

### 1. What is C?

**Answer:**

C is a general-purpose, procedural programming language developed by Dennis Ritchie at Bell Labs in 1972. It provides low-level memory access, making it suitable for system programming, embedded systems, operating systems, and device drivers.

---

### 2. What are the features of C?

**Answer:**

- Simple and efficient
  - Portable
  - Structured programming
  - Rich set of operators
  - Direct memory access through pointers
  - Fast execution
  - Supports modular programming
  - Suitable for embedded and system software
- 

### 3. What is the difference between C and C++?

**Answer:**

| C                       | C++                  |
|-------------------------|----------------------|
| Procedural              | Object-Oriented      |
| No classes              | Supports classes     |
| No inheritance          | Supports inheritance |
| No function overloading | Supports overloading |
| Uses malloc/free        | Uses new/delete      |

**Interview Tip:**

C focuses on functions and data, while C++ focuses on objects.

---

#### 4. What are keywords in C?

##### **Answer:**

Keywords are reserved words that have predefined meanings and cannot be used as identifiers.

Examples:

int  
char  
return  
if  
while  
for  
static  
const

---

#### 5. What is a token in C?

##### **Answer:**

The smallest meaningful unit in a C program.

Types:

- Keywords
- Identifiers
- Constants
- Operators
- Strings
- Special symbols

Example:

```
int x = 10;
```

Tokens:

```
int  
x  
=
```

10

;

---

## **6. What is the difference between declaration and definition?**

### **Declaration:**

Tells compiler that something exists.

```
extern int count;
```

### **Definition:**

Allocates memory.

```
int count = 0;
```

### **Interview Answer:**

Declaration introduces a variable; definition creates storage for it.

---

## **7. What is scope in C?**

### **Answer:**

Scope defines where a variable can be accessed.

Types:

### **Local Scope**

```
void fun()  
{  
    int x;  
}
```

### **Global Scope**

```
int x;
```

Accessible throughout the file.

---

## **8. What are storage classes in C?**

### **Answer:**

Storage classes define visibility and lifetime.

**auto**

auto int x;

Default local variable.

**static**

Retains value between function calls.

**extern**

Variable defined elsewhere.

**register**

Suggests storing variable in CPU register.

---

**9. What is the difference between local and global variables?****Local**

Declared inside function

Limited scope

Created on function entry

Stored in stack

**Global**

Declared outside function

File scope

Exists entire program

Stored in data segment

---

**10. What is the lifetime of a variable?****Answer:**

Lifetime is the duration for which memory exists.

Example:

```
void fun()
{
    int x;
}
```

x exists only while function executes.

Static variables live throughout program execution.

---

### 11. What are the basic data types in C?

**Answer:**

char  
int  
float  
double  
void

Modifiers:

short  
long  
signed  
unsigned

---

### 12. What is the size of int, char, float, and double?

**Answer:**

Typical sizes:

| Type   | Size    |
|--------|---------|
| char   | 1 byte  |
| int    | 4 bytes |
| float  | 4 bytes |
| double | 8 bytes |

**Note:**

Sizes are compiler-dependent.

Use:

sizeof(type)

---

### 13. What is the difference between signed and unsigned?

**Signed:**

Stores positive and negative values.

signed int x;

**Unsigned:**

Stores only positive values.

unsigned int x;

Example:

signed int:

-2147483648 to 2147483647

unsigned int:

0 to 4294967295

---

**14. What happens when signed and unsigned variables are mixed?**

**Answer:**

Signed values are usually converted to unsigned.

Example:

int a = -1;

unsigned int b = 1;

if(a > b)

Result may surprise you because a becomes a very large unsigned value.

**Common Interview Pitfall.**

---

**15. What is integer promotion?**

**Answer:**

Small integer types are automatically promoted to int before arithmetic.

Example:

char a = 10;

char b = 20;

a + b

Both operands become int.

---

**16. What is type casting?****Answer:**

Converting one data type into another explicitly.

Example:

```
float x = 10.5;
```

```
int y = (int)x;
```

Output:

10

Fractional part is discarded.

---

**17. What is implicit conversion?****Answer:**

Automatic type conversion performed by compiler.

Example:

```
int a = 5;
```

```
float b = a;
```

Compiler converts integer to float automatically.

---

**18. What is the difference between ++i and i++?****Pre-increment**

```
++i
```

Increment first, then use value.

**Post-increment**

```
i++
```

Use value first, then increment.

Example:

```
int i = 5;
```

```
printf("%d", ++i);
```

Output:

6

Example:

```
int i = 5;
```

```
printf("%d", i++);
```

Output:

5

---

## 19. What is operator precedence?

**Answer:**

Determines which operator executes first.

Example:

$2 + 3 * 4$

Result:

14

because:

$3 * 4$

executes first.

---

## 20. What is associativity?

**Answer:**

Determines evaluation order when operators have same precedence.

Example:

$10 - 5 - 2$

Evaluated:



$(10 - 5) - 2$

Result:

3

because subtraction is left-to-right associative.

---

## 21. Difference between while and do-while?

### **while**

```
while(condition)
{
}
```

Condition checked first.

### **do-while**

```
do
{
}
while(condition);
```

Condition checked after execution.

### **Important:**

do-while executes at least once.

---

## 22. When should switch be used?

### **Answer:**

Use switch when selecting among multiple fixed choices.

Example:

```
switch(choice)
{
case 1:
case 2:
}
```

Cleaner than many if-else statements.

---

### 23. Can switch use floating-point values?

#### Answer:

No.

Valid:

char

int

enum

Invalid:

float

double

---

### 24. What happens if break is omitted in switch?

#### Answer:

Execution falls through to next case.

Example:

```
switch(x)
{
case 1:
    printf("One");

case 2:
    printf("Two");
}
```

For x=1:

Output:

OneTwo

---

### 25. Difference between continue and break?

#### break

Exits loop completely.

```
for(...)
{
    break;
}
```

### **continue**

Skips remaining statements of current iteration.

```
for(...)
{
    continue;
}
```

### **Interview Shortcut:**

- break → leave loop
- continue → skip current iteration

## **26. What is the purpose of goto?**

### **Answer:**

goto transfers program control directly to a labeled statement.

```
goto error;
```

```
error:
    printf("Error");
```

### **When is it used?**

- Error handling
- Exiting deeply nested loops
- Resource cleanup in embedded/Linux kernel code

### **Interview Note**

Avoid excessive use because it makes code difficult to maintain.

---

## **27. Can goto jump between functions?**

### **Answer:**

No.

goto can only jump within the same function.

Invalid:

```
void fun1()
{
    goto label; // Error
}
```

```
void fun2()
{
label:
}
```

---

## 28. What is the dangling else problem?

**Answer:**

Occurs when an else can be associated with more than one if.

Example:

```
if(a)
    if(b)
        printf("X");
    else
        printf("Y");
```

The else belongs to the nearest unmatched if.

Equivalent to:

```
if(a)
{
    if(b)
        printf("X");
    else
        printf("Y");
}
```

---

## 29. Explain nested loops.

**Answer:**

A loop inside another loop.

Example:

```
for(i=0;i<3;i++)
{
    for(j=0;j<2;j++)
    {
        printf("*");
    }
}
```

### **Complexity**

Outer loop × Inner loop

Example:

$N \times M$

---

## **30. How do you exit multiple loops?**

### **Method 1: Flag**

```
int stop = 0;
```

```
for(...)
{
    for(...)
    {
        if(condition)
        {
            stop = 1;
            break;
        }
    }
}
```

```
if(stop)
    break;
}
```

### **Method 2: goto**

```
goto exit_loops;
```

```
exit_loops:
```

Common in embedded code.

---

### **31. What is function prototyping?**

#### **Answer:**

A function declaration that informs the compiler about:

- Function name
- Return type
- Parameters

Example:

```
int add(int, int);
```

---

### **32. Why are function declarations important?**

#### **Answer:**

They allow the compiler to:

- Check parameter types
- Check return types
- Detect errors before linking

Without a prototype:

```
add(10,20);
```

Compiler may assume incorrect types.

---

### **33. What is recursion?**

#### **Answer:**

A function calling itself.

Example:

```
int fact(int n)
{
    if(n==0)
        return 1;

    return n * fact(n-1);
}
```

---

### **34. Advantages and disadvantages of recursion?**

#### **Advantages**

- Cleaner code
- Natural for tree algorithms
- Easy to understand

#### **Disadvantages**

- More memory usage
  - Function call overhead
  - Stack overflow risk
- 

### **35. What is tail recursion?**

#### **Answer:**

Recursive call is the last statement executed.

Example:

```
int fun(int n, int result)
{
    if(n==0)
        return result;

    return fun(n-1, result*n);
}
```

Some compilers can optimize tail recursion.

---

### **36. How are arguments passed in C?**

**Answer:**

C passes arguments by value.

Example:

```
void fun(int x)
{
    x = 100;
}
```

```
int a = 10;
```

```
fun(a);
```

a remains unchanged.

---

**37. Is C pass-by-value or pass-by-reference?****Answer:**

C is strictly pass-by-value.

Even pointers are passed by value.

Example:

```
void fun(int *p)
```

The pointer itself is copied.

---

**38. How do you simulate pass-by-reference?****Answer:**

Use pointers.

```
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Call:

```
swap(&x, &y);
```



---

### 39. Can a function return an array?

#### Answer:

No.

Invalid:

```
int[] getArray()
```

#### Alternatives

Return:

- Pointer
- Structure
- Dynamically allocated memory

Example:

```
int *getArray(void)
{
    static int arr[5];
    return arr;
}
```

---

### 40. Can a function return a pointer?

#### Answer:

Yes.

```
int *fun()
{
    static int x = 10;
    return &x;
}
```

#### Pitfall

Never return address of local variable.

Wrong:

```
int *fun()
{
    int x = 10;
    return &x;
}
```

After function exits, x no longer exists.

---

#### 41. Difference between array and linked list?

| Array                | Linked List          |
|----------------------|----------------------|
| Contiguous memory    | Non-contiguous       |
| Fast random access   | Sequential access    |
| Fixed size           | Dynamic size         |
| Less memory overhead | Extra pointer memory |

---

#### 42. How are arrays stored in memory?

**Answer:**

Contiguous memory locations.

Example:

```
int arr[4];
```

Memory:

1000

1004

1008

1012

(assuming 4-byte integers)

---

#### 43. Why does array indexing start from 0?

**Answer:**

Array access is defined as:

`arr[i]`

Equivalent to:

`*(arr + i)`

For first element:

`*(arr + 0)`

No offset calculation required.

---

#### **44. What is a multidimensional array?**

**Answer:**

Array containing arrays.

Example:

```
int matrix[3][4];
```

Represents:

3 rows

4 columns

---

#### **45. Difference between `arr` and `&arr`?**

Example:

```
int arr[10];
```

**`arr`**

Type:

`int *`

Points to first element.

**`&arr`**

Type:

`int (*)[10]`

Pointer to entire array.

---

#### 46. Can array size be changed at runtime?

**Answer:**

Normal arrays cannot.

```
int arr[10];
```

Size is fixed.

Alternatives:

- malloc()
  - realloc()
  - VLA (C99)
- 

#### 47. What is a Variable Length Array (VLA)?

**Answer:**

Array size determined during runtime.

```
int n;  
scanf("%d",&n);
```

```
int arr[n];
```

Introduced in C99.

**Limitation**

Stored on stack.

Large VLAs can cause stack overflow.

---

#### 48. Why can't arrays be assigned directly?

Invalid:

```
int a[5];  
int b[5];
```

```
a = b;
```

**Reason**

Array names are not modifiable lvalues.

Correct:

```
memcpy(a, b, sizeof(a));
```

---

#### **49. What is array decay?**

**Answer:**

Array automatically converts to pointer to first element.

Example:

```
int arr[5];
```

```
fun(arr);
```

Actually becomes:

```
fun(&arr[0]);
```

#### **Interview Favorite**

Arrays lose size information when passed to functions.

---

#### **50. Difference between sizeof(arr) and sizeof(ptr)?**

Example:

```
int arr[10];  
int *ptr = arr;
```

**sizeof(arr)**

```
sizeof(arr)
```

Output:

40

(10 × 4)

**sizeof(ptr)**

```
sizeof(ptr)
```

Output:

8

(on 64-bit system)

### **Interview Trap**

```
void fun(int arr[])
{
    printf("%zu", sizeof(arr));
}
```

Output:

8

NOT 40.

Because arr becomes a pointer parameter.

---

### **Top Interview Traps (26–50)**

#### **Trap 1**

```
int arr[5];
```

```
printf("%zu", sizeof(arr));
```

Returns total bytes, not element count.

---

#### **Trap 2**

```
void fun(int arr[])
{
    printf("%zu", sizeof(arr));
}
```

Returns pointer size.

---

#### **Trap 3**

```
int *fun()
{
    int x = 10;
    return &x;
}
```

Returns dangling pointer.

---

#### **Trap 4**

```
int arr[5];  
arr++;
```

Compilation error.

Array names are not modifiable.

---

#### **Trap 5**

```
arr[i]
```

is exactly:

```
*(arr + i)
```

Therefore:

```
i[arr]
```

also works (rare interview trick).

### **51. Difference between char str[] and char \*str?**

#### **Array**

```
char str[] = "Hello";
```

Memory:

```
H e l l o \0
```

Stored in writable memory.

---

#### **Pointer**

```
char *str = "Hello";
```

str points to a string literal.

String literals should be treated as read-only.

---

## Interview Answer

`char str[]` creates a copy of the string, while `char *str` points to a string literal.

---

### 52. What is a string literal?

#### Answer:

A sequence of characters enclosed in double quotes.

Example:

"Hello"

Compiler automatically adds:

`'\0'`

Memory:

H e l l o \0

---

### 53. Why are string literals often read-only?

#### Answer:

Most implementations place string literals in read-only memory.

Example:

```
char *s = "Hello";
```

```
s[0] = 'h';
```

Result:

Undefined Behavior

May crash with a segmentation fault.

---

### 54. Difference between `strcpy()` and `strncpy()`?

#### `strcpy()`

Copies entire string.

```
strcpy(dest, src);
```



---

**strncpy()**

Copies specified number of characters.

```
strncpy(dest, src, n);
```

---

**Pitfall**

```
strncpy(dest, src, n);
```

may not append '\0'.

---

**55. Difference between strcmp() and strncmp()?****strcmp()**

Compares entire strings.

```
strcmp(a, b);
```

---

**strncmp()**

Compares first n characters.

```
strncmp(a, b, n);
```

Example:

```
strncmp("Hello", "Help", 3);
```

returns:

0

because first 3 characters match.

---

**56. What does strlen() return?****Answer:**

Returns the number of characters before the null terminator.

Example:

```
strlen("Hello");
```

Output:

5

---

### 57. Why doesn't strlen() include '\0'?

Because '\0' marks the end of the string and is not considered part of the string content.

Example:

```
char str[] = "ABC";
```

Memory:

A B C \0

Length:

3

---

### 58. Common causes of buffer overflow?

#### Copying too much data

```
char buf[5];  
strcpy(buf, "Hello World");
```

---

#### Invalid indexing

```
buf[10] = 'A';
```

---

#### Unsafe input

```
gets(buf);
```

---

### 59. Why is gets() dangerous?

#### Answer:

It does not check buffer size.

Example:

```
char buf[10];  
gets(buf);
```

Long input can overwrite memory.

For this reason, gets() was removed from the C standard.

---

## **60. Safe alternatives to gets()?**

### **Preferred**

```
fgets(buffer, sizeof(buffer), stdin);
```

### **Example**

```
char buf[20];  
  
fgets(buf, sizeof(buf), stdin);
```

---

## **61. What is a pointer?**

### **Answer:**

A variable that stores the address of another variable.

Example:

```
int x = 10;  
  
int *p = &x;
```

Memory:

x = 10

p = address of x

---

## **62. What is a NULL pointer?**

### **Answer:**

A pointer that points to no valid object.

```
int *p = NULL;
```

Used to indicate:

No address assigned

---

### **63. Difference between NULL and void \*?**

#### **NULL**

Represents an invalid pointer.

```
int *p = NULL;
```

---

#### **void \***

Generic pointer type.

```
void *p;
```

Can store address of any data type.

---

### **64. What is a dangling pointer?**

#### **Answer:**

A pointer that references memory that is no longer valid.

Example:

```
int *p;
```

```
{  
    int x = 10;  
    p = &x;  
}
```

After block exits:

p

is dangling.

---

### **65. What is a wild pointer?**

#### **Answer:**

An uninitialized pointer.

```
int *p;
```

Value is garbage.

Dereferencing it causes undefined behavior.

---

## **66. What is a void pointer?**

### **Answer:**

A generic pointer capable of holding addresses of any type.

```
void *ptr;
```

Example:

```
int x = 10;
```

```
void *ptr = &x;
```

Before dereferencing:

```
*(int *)ptr
```

must be cast.

---

## **67. Can pointer arithmetic be performed on void \*?**

### **Standard C Answer:**

No.

```
void *p;
```

```
p++;
```

is invalid according to the C standard.

Reason:

Compiler does not know object size.

---

## **68. Difference between pointer and array?**

**Array**

Fixed size

Owns storage

Cannot be reassigned

sizeof() gives full size

**Pointer**

Variable

References storage

Can be reassigned

sizeof() gives pointer size

---

**69. What is a pointer to pointer?****Answer:**

A pointer that stores the address of another pointer.

Example:

```
int x = 10;
```

```
int *p = &x;
```

```
int **pp = &p;
```

---

**Access**

\*pp

gives:

p

\*\*pp

gives:

x

---

**70. What is a function pointer?****Answer:**

A pointer that stores the address of a function.

Example:

```
int add(int a, int b);
```

```
int (*fp)(int, int);
```

Assignment:

```
fp = add;
```

Call:

```
fp(2,3);
```

---

## **71. How are function pointers used?**

### **Callbacks**

```
qsort()
```

### **Interrupt handlers**

### **Driver frameworks**

### **State machines**

### **Event systems**

Very common in embedded systems.

---

## **72. What is pointer arithmetic?**

### **Answer:**

Arithmetic performed on addresses.

Example:

```
int arr[5];
```

```
int *p = arr;
```

```
p++;
```

moves to next integer.

---

### **Example**

If:

p = 1000

and:

sizeof(int)=4

then:

p++;

becomes:

1004

---

### 73. Why is ptr++ dependent on data type?

Because compiler increments by:

sizeof(type)

Example:

char \*c;

Increment:

+1

---

int \*i;

Increment:

+4

(typically)

---

### 74. Difference between \*ptr++ and (\*ptr)++?

#### Case 1

\*ptr++

Equivalent to:

\*(ptr++)

Pointer moves.



Value unchanged.

---

## **Case 2**

`(*ptr)++`

Pointer unchanged.

Value incremented.

---

## **Interview Favorite**

Know operator precedence.

---

### **75. Explain:**

#### **a) `const int *p`**

Pointer to constant integer.

`const int *p = &x;`

Allowed:

`p = &y;`

Not allowed:

`*p = 10;`

---

#### **b) `int *const p`**

Constant pointer.

`int *const p = &x;`

Allowed:

`*p = 10;`

Not allowed:

`p = &y;`

---

### **c) const int \*const p**

Constant pointer to constant integer.

Not allowed:

```
*p = 10;
```

```
p = &y;
```

Neither the pointer nor the value can change.

---

## **Top Pointer Interview Traps (51–75)**

### **Trap 1**

```
char *s = "Hello";
```

```
s[0] = 'h';
```

**Undefined Behavior**

---

### **Trap 2**

```
int *p;
```

```
printf("%d", *p);
```

Wild pointer.

---

### **Trap 3**

```
int *p = NULL;
```

```
*p = 10;
```

NULL dereference.

---

### **Trap 4**

```
int *p;
```

```
free(p);
```

Undefined behavior if p was never allocated.

---

#### **Trap 5**

```
int *p = malloc(sizeof(int));
```

```
free(p);
```

```
*p = 10;
```

Use-after-free.

---

#### **Trap 6**

```
int arr[5];
```

```
printf("%d", *(arr+2));
```

Equivalent to:

```
arr[2]
```

---

#### **Trap 7**

```
arr[i]
```

and

```
i[arr]
```

are identical.

---

#### **Trap 8**

```
int *p = NULL;
```

```
free(p);
```

Perfectly safe.

---

#### **Trap 9**

```
void *p = &x;
```

```
printf("%d", *p);
```

Compilation error.

Must cast first.

---

### **Trap 10**

```
int *p = &x;
```

```
int **pp = &p;
```

```
**pp
```

accesses x.

### **76. Can NULL be dereferenced?**

**Answer:**

No.

Example:

```
int *p = NULL;
```

```
*p = 10;
```

Result:

Undefined Behavior

Usually causes a segmentation fault or hard fault.

---

### **77. What is pointer aliasing?**

**Answer:**

Multiple pointers referring to the same memory location.

Example:

```
int x = 10;
```

```
int *p1 = &x;
```

```
int *p2 = &x;
```

Both pointers access the same object.

### Importance

Compilers perform optimization based on aliasing assumptions.

---

### 78. What are double pointers used for?

#### Answer:

Common uses:

#### Dynamic 2D arrays

```
int **matrix;
```

#### Modifying pointers inside functions

```
void allocate(int **p)
{
    *p = malloc(sizeof(int));
}
```

#### Linked lists

```
Node **head;
```

---

### 79. Difference between stack and heap?

| Stack                | Heap                  |
|----------------------|-----------------------|
| Automatic allocation | Dynamic allocation    |
| Fast                 | Slower                |
| Managed by compiler  | Managed by programmer |
| Limited size         | Larger                |
| Auto cleanup         | Must free manually    |

---

### 80. What is malloc()?

#### Answer:

Allocates memory dynamically.

```
int *p = malloc(10 * sizeof(int));
```

### **Characteristics**

- Uninitialized memory
  - Returns void\*
  - Returns NULL on failure
- 

## **81. Difference between malloc() and calloc()?**

### **malloc()**

```
malloc(n * sizeof(int));
```

Memory contents:

Garbage values

---

### **calloc()**

```
calloc(n, sizeof(int));
```

Memory contents:

All bytes initialized to zero

---

## **82. Difference between malloc() and realloc()?**

### **malloc()**

Creates new memory.

```
malloc(100);
```

---

### **realloc()**

Resizes existing memory.

```
realloc(ptr, 200);
```

### **Benefits**

- Grow memory

- Shrink memory
  - Preserve existing data
- 

### **83. What does free() do?**

#### **Answer:**

Returns dynamically allocated memory to the heap.

```
free(ptr);
```

#### **Best Practice**

```
free(ptr);
```

```
ptr = NULL;
```

Avoids accidental reuse.

---

### **84. What is a memory leak?**

#### **Answer:**

Allocated memory that is no longer accessible.

Example:

```
int *p = malloc(100);
```

```
p = NULL;
```

Original memory cannot be freed.

Memory leak occurs.

---

### **85. What is heap fragmentation?**

#### **Answer:**

Heap becomes divided into many small free blocks.

Example:

```
[Used][Free][Used][Free][Used]
```

Large allocations may fail even when total free memory is sufficient.

Common concern in embedded systems.

---

### **86. What happens when malloc() fails?**

**Answer:**

Returns:

NULL

Example:

```
int *p = malloc(1000);
```

```
if(p == NULL)
{
    // handle error
}
```

---

### **87. Why check malloc() return value?**

**Answer:**

Dereferencing NULL causes a crash.

Wrong:

```
int *p = malloc(100);
```

```
*p = 10;
```

Correct:

```
if(p != NULL)
{
    *p = 10;
}
```

---

### **88. What is use-after-free?**

**Answer:**

Accessing memory after freeing it.

Example:



```
int *p = malloc(sizeof(int));
```

```
free(p);
```

```
*p = 10;
```

Result:

Undefined Behavior

Very common source of bugs.

---

## **89. Difference between memory leak and dangling pointer?**

### **Memory Leak**

Memory exists but pointer is lost.

```
p = malloc(100);
```

```
p = NULL;
```

---

### **Dangling Pointer**

Pointer exists but memory is invalid.

```
free(p);
```

Pointer still points to released memory.

---

## **90. What is double free?**

### **Answer:**

Freeing the same memory twice.

Example:

```
free(ptr);
```

```
free(ptr);
```

Result:

Undefined Behavior

May corrupt heap structures.

---

## 91. Difference between structure and union?

### Structure

Each member gets separate storage.

```
struct S
{
    int a;
    char b;
};
```

---

### Union

All members share same memory.

```
union U
{
    int a;
    char b;
};
```

Only one member is valid at a time.

---

## 92. Why does structure padding occur?

### Answer:

To satisfy alignment requirements and improve CPU access efficiency.

Example:

```
struct S
{
    char c;
    int i;
};
```

Layout:

```
c
pad
pad
```

pad

i

Size:

8 bytes

not 5.

---

### 93. How can padding be reduced?

#### Reorder members

Bad:

```
struct S
{
    char c;
    int i;
    char d;
};
```

---

Better:

```
struct S
{
    int i;
    char c;
    char d;
};
```

#### Compiler packing

```
#pragma pack(1)
```

Use carefully.

---

### 94. What is structure alignment?

**Answer:**

Memory addresses are aligned according to data type requirements.

Example:

int

typically aligned to:

4-byte boundary

Benefits:

- Faster access
  - Fewer CPU cycles
- 

### 95. Can structures be compared using == ?

**Answer:**

No.

Invalid:

```
struct S a, b;
```

```
if(a == b)
```

Compilation error.

**Alternative**

Compare members individually.

```
if(a.x == b.x &&  
   a.y == b.y)
```

---

### 96. Can structures contain themselves?

**Invalid**

```
struct Node  
{  
    struct Node next;  
};
```

Infinite size.

---

**Valid**

```
struct Node
{
    struct Node *next;
};
```

Used in linked lists.

---

## 97. Difference between . and -> operators?

### Dot Operator

Access structure member directly.

emp.salary

---

### Arrow Operator

Access through pointer.

ptr->salary

Equivalent:

(\*ptr).salary

---

## 98. What is a bit-field?

### Answer:

Allows control over the number of bits used by a member.

Example:

```
struct Status
{
    unsigned ready : 1;
    unsigned error : 1;
    unsigned mode : 2;
};
```

### Common Uses

- Hardware registers
- Communication protocols

- Embedded systems
- 

### 99. When should a union be used?

#### Answer:

When multiple data types share the same memory.

Example:

```
union Data
{
    int i;
    float f;
};
```

#### Benefits

- Saves memory
  - Useful in protocol parsing
  - Hardware register access
- 

### 100. How do you calculate structure size?

Example:

```
struct S
{
    char c;
    int i;
};
```

Calculation:

char = 1 byte  
padding = 3 bytes  
int = 4 bytes

Total = 8 bytes

Verify:

```
printf("%zu", sizeof(struct S));
```